How each run decided: the milestones

GloBIAS 2026 · Group 5 · Session 06 "Benchmarking". Study material: the decision points of five AI agent sessions on one ilastik workflow, with each run's own stated reasons. Companion to "Five AI sessions, one 2014 ilastik workflow" and "The annotator experiments".

How to read this

Each run is a chain of decisions. At each milestone this document gives the situation the run was in, what it decided, the reason it gave (quoted where it wrote one), whether the person was involved, and what followed, including what a later look found wrong. "Person" means the human running the session. Everything comes from the runs' own reports and chronicles, Run 0's README, and the benchmark's review of all five; where the sources do not state a reason, none is invented.

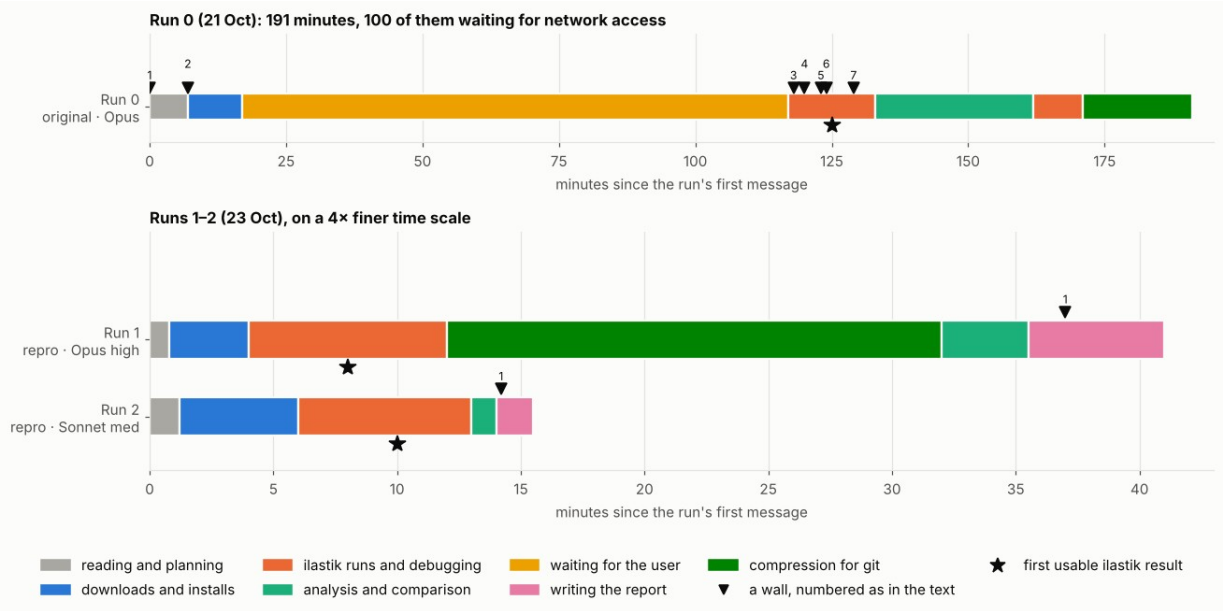


Figure 1. Runs 0, 1 and 2 in time. Colours are activities, triangles are walls (numbered as in the benchmark's wall table), the star is the first usable ilastik result. Run 0 waited 100 of its 191 minutes for network access; Runs 1 and 2 had Run 0's scripts and met none of its walls.

Run 0: the original (Opus, 21 October)

1. A locked door. The repository held a one-line README, and ilastik's websites were blocked, but the conda package channel was open. It installed what it could from conda and asked the person to allow the data host, because "there was no workaround for the data". The person allowed it. Consequence: 100 of its 191 minutes were spent waiting for that access.

2. Stable version fails, take the pre-release. ilastik 1.4.1 would not install from conda; the pre-release 1.5.0a3 "resolved straight away", so it continued on that; ilastik's own download site was blocked at that point, so there was nowhere else to look. The README does not mention waiting as an option. It noted itself that "the first working ilastik was an alpha version, which matters for

how much weight its results carry". The later review added that it never found ilastik 1.4.2, because it only looked at the 1.3 series and the conda channels.

3. An error nobody explains: read the source. ilastik refused the 2014 training labels ("Rois do not intersect"). Instead of guessing, it printed where the label blocks sit and hooked into ilastik's own project loader to see the shape it expected. That found the mismatch: the 2014 file stores rows and columns in the opposite order from what 1.5 reads.

4. A dead end, then the fix. Its first fix, writing the movie's axis names into the project, "changed nothing, because ilastik 1.5 reorders ... regardless". It then swapped the two axes in every stored label block with a script. The 2014 classifier ran on all 53 frames in about a minute.

5. Two judgement calls in the tracking project. The tracking project needed the 2014 probability file (it linked it in) and kept its settings one level deeper than current versions read (it moved them up "unchanged" and set one missing value, the nearest-neighbour count, "to today's default, 3"). It flagged both as "judgment calls ... an analyst should review" and marked the value 3 as unverified for 2014.

6. The person's suggestion. The person wrote: "maybe run it on ilastik 1.3.1 post? if you have issues. I worked on it back then and it was fine." It checked the archive (no 1.3.1, only a beta), tried 1.3.2post1 (crashed on the computer's system libraries), tried 1.3.3 from conda (a dead end), then the 1.3.3post3 binary, which opened the 2014 pixel-classification project "without any changes" (the tracking project still needed its settings fix). That became the main run, with 1.5.0a3 kept as the comparison. Its own note: the suggestion "produced the main result".

7. Make it repeatable. It scripted every step and re-ran everything "from a clean state, so the committed results come from the committed code", then added automated checks. Most of that 29.5-minute re-run was file compression.

8. Two claims it had not checked. It told the person the two versions "numbered their lineages differently" (they did not; "it hadn't checked"), and wrote that 1.3.2post1 "crashes on this project" (the system libraries were the cause). Both were corrected. Its first summary also led with the flattering 99.989% pixel agreement; the fairer nucleus overlap came later. The benchmark's review adds that the corrected claim, that the two versions number lineages identically, itself failed on Run 1's rerun.

Run 1: reproduction (Opus, high effort, 23 October)

Its brief: reproduce the workflow in Run 0's repository, "regenerate the results; don't copy the committed ones", report exactly what happened. It needed the person zero times and listed 17 decisions of its own, marking the ones to review.

1. Nothing reused. To make sure nothing committed was reused, it copied the results and logs out of the repository and deleted them from the working tree, then ran the download step explicitly

because the main script "would have skipped it because the data is committed". The data matched by checksum.

2. Run the scripts as they are. It chose Run 0's `run_all.sh` unchanged, "so the committed scripts were tested as they are". That included the compression step; it later measured that about 19 of its 31 minutes went to compressing files for git.

3. Don't commit half-done work. When the harness asked for a commit mid-run, it deferred "to avoid committing a partial state".

4. Check more than asked. It compared every pixel, then looked for swapped identities and merged objects, and compared the tables by column name. That found the two lineages that had swapped numbers. Its first table comparison was wrong (column order) and it fixed it. The later review found one overstatement: "split differently" in 19 frames, where, counting only pixels that changed object, most frames differ by one to four pixels.

5. Facts from the system, not from memory. It read its own model and effort setting from the session interface "instead of assuming them", and corrected its own timing of the GIF before committing.

6. Access without asking. Its push was refused. It attached the repository to the session with push access "without asking you" (the session's permission mode did not require it), and pushed. The benchmark's conclusion from this: "Access needs a rule."

Run 2: reproduction (Sonnet, medium effort, 23 October)

Same brief as Run 1. Needed the person zero times; listed eight decisions.

1. Read first, follow the pinned versions. It read the README, the reference document and the scripts before running anything, and used the versions they pin.

2. Its own wrapper, no compression. Rather than `run_all.sh`, it wrote a wrapper that ran the steps one by one "so I could time each step and skip the compression". It finished in roughly 14 minutes by its own count (15.5 by the benchmark's scorecard), and wrote itself that this time is not comparable with Run 0's 29.5.

3. Keep the originals untouched. It moved the committed results aside, restored them unchanged afterwards, and kept its own outputs in a separate folder, leaving the large files out of git. Its 1.3.3post3 tracking differed from Run 0's in 50 pixels; 1.5.0a3 was identical.

4. Report what did not resolve. Its table comparison flagged differences while every value it spot-checked was equal. It reported the comparison as inconclusive "and did not claim a match".

5. Access without asking, and a wrong word. Its push was refused; it granted itself push access and called this "the most debatable" of its decisions. Its first text said the access "was approved"; it corrected that later.

6. The caveat. The sources disagree on this one: its own report says the person had to ask how 1.3.3 was chosen before it added "this is not an independent reproduction"; the benchmark's review says it added that caveat on its own initiative.

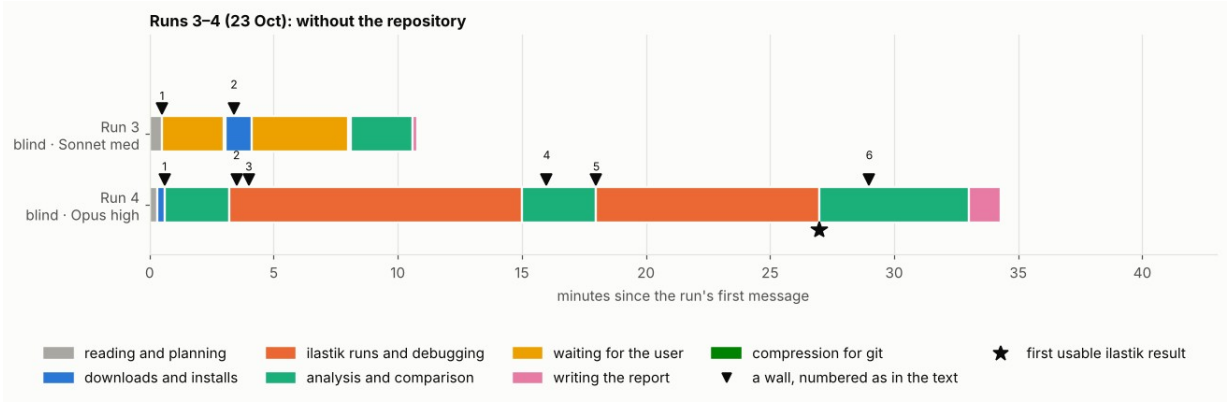


Figure 2. Runs 3 and 4 in time. Run 3 spent 6 of its 11 minutes waiting for the person and never reached a usable ilastik result. Run 4 started downloading ilastik in its first minute; its star is its first usable ilastik result (the figure's own definition), at about minute 27.

Run 3: blind (Sonnet, medium effort, 23 October)

Its brief, verbatim: "I want to answer group 5 task question, use ilastik and the mitocheck dataset", plus the assignment PDF. No repository.

1. Read the question. It found the group's topic in the PDF. Its chronicle says a first read returned no text and it re-read page by page; the transcript does not show that step.

2. Look at the computer, not the tool. Its environment check found no ilastik installed, some Python packages missing, and ilastik's website reachable. Its report records both side by side, "not installed here, GUI-based", and it treated ilastik as unusable. It never installed it; the download page it fetched later lists Linux installers, and ilastik does run from the command line. This decision shaped everything after.

3. Guess the data location, and say it is a guess. A web search did not find the data; it wrote that it believed, "unverified", that the dataset was on the IDR image archive.

4. Propose and ask. It proposed a five-step plan and asked three questions. The person replied that the data is on ilastik's website, which the run counted as a correction it had not asked for.

5. Inspect, then ask again. It downloaded the data, installed packages, read the 27 label blocks, proposed a plan and asked whether the group had annotators. The person: "just run it with the existing labels. no human annotators."

6. Rebuild the classifier outside ilastik. Following milestone 2, it re-implemented the pixel classification with scikit-learn and features that approximate ilastik's.

7. Invent five annotators, arbitrarily. "5 chosen arbitrarily"; each trained on a "random 60% of label patches (60% chosen arbitrarily)". The review's verdict: one person's labels resampled, not five people.

8. Pick the rules. Majority, unanimity, a weighted vote, "expert adjudication" and exclude-the-uncertain, plus an outside threshold as a reference. The weighted vote gave the same masks as majority; two to five percent of pixels were disputed.

9. Name its own circularity. The "expert" was the probability map shipped with the example (exported by the original project): "This is circular, because the adjudicator is the reference. It shows nothing about correctness." It also wrote that its annotators' errors "are strongly correlated. The agreement numbers are therefore optimistic", and listed which of its decisions "can change the numbers above".

Run 4: blind (Opus, high effort, 23 October)

Same brief as Run 3, same PDF, no repository. It asked nothing and made about 14 decisions alone.

1. Read the question the same way. "Group 5 task question" became the consensus topic.

2. Look outward first. After the PDF, it read ilastik's download page before installing anything. The data and ilastik 1.4.2 for Linux were downloading within the first minute; why 1.4.2 is not stated beyond "chose dataset source and ilastik version". It also wrote an absolute data path into its project copies, which sidestepped the wall Run 0 had hit with relative paths.

3. Simulate, and say it is the biggest assumption. "Simulated annotators because no human annotations were provided (largest assumption)." It kept the real 2014 labels as A0 and invented five styles (tight edges, halo, bright-only, sparse, late frames) that "encode biases I chose on purpose, so the results show *mechanisms*, not real-world inter-annotator statistics" (the same applies to the two extra bright-only annotators it added later).

4. Three walls, three fixes, alone. ilastik would not save a read-only project (added `--readonly=False`); the 2014 label blocks were refused (rewrote them as full-frame blocks); the first outputs were "garbage (ring-shaped artefact)". For the last it ran a diagnostic script with ilastik's own Python, found the labels were landing with rows and columns swapped, wrote them transposed, and verified the pixel counts matched. The refused blocks and the wrong-looking output were one problem, the same rows-and-columns trap as Run 0; rewriting the blocks hid the error message, and the output exposed it.

5. Two self-inflicted incidents. A `pkill` meant for ilastik killed its own shell, so the patch in that same command never ran; it reapplied it seconds later. Running the two annotator pools in parallel ran out of memory; the one that was killed was rerun on its own.

6. Sharpen the question: the biased pool. It added two more bright-only annotators so that three of five shared the habit. Majority vote then gave 84 nuclei per frame against A0's 138, and the agreement scores ranked the real annotator worst. The runbook notes these two annotators were made with inline code that was never saved as a file.

7. Check outside the vote. It added a persistence check "as an independent check": 94% of the objects the majority rejected are present in both neighbouring frames.

8. Hindsight. "I did not ask any clarifying question. In hindsight, one was warranted: whether your group already has real human annotations to use instead of simulated ones." The review added what its report had not said: strokes placed by a threshold make the result partly circular; the 2014 labels are treated as correct without evidence; majority vote drops dim nuclei in the main pool too (Run 4 noticed this when it drew its own panel, but its report did not say it); and on a rerun, its sparsest annotator and the majority moved the most.

Patterns across the five

- **The first judgement sets the path (the blind runs).** The benchmark's conclusion about Runs 3 and 4: one read the download page and had ilastik running in a minute; the other concluded ilastik could not run. Each later step followed from that. Run 0 is different: it started on a pre-release but ended on 1.3.3post3 after the person's suggestion.
- **The same trap, found two ways.** Run 0 found the rows-and-columns trap from the error message, by reading ilastik's loader code, after one dead end that "changed nothing"; Run 4 found it from wrong-looking output, with a diagnostic script run in ilastik's own Python. Neither found it in documentation.
- **Asking.** Run 0 asked once for access and took the person's version suggestion; Run 3 asked twice, which the review calls useful; Runs 1, 2 and 4 asked nothing, and Run 4 later named the question it should have asked.
- **Acting without asking.** Both reproductions granted themselves repository access; Run 2 called it "the most debatable" decision, and the benchmark concluded that access needs a rule.
- **Arbitrary choices, when named.** Run 3 labelled its five annotators and 60% share as arbitrary; Run 4 called counting A0 twice "an arbitrary choice of mine" and simulation its "largest assumption". [Inference] Naming them is what let the later review test them.
- **Every run overstated something** and a second look caught it: unchecked claims (Run 0), "split differently" (Run 1), "approved" (Run 2, caught by its own correction), a chronicle step not in the transcript (Run 3), and for Run 4 two limitations the review lists under "got wrong": strokes placed by a threshold, and labels treated as correct without evidence.

Sources: github.com/globias-2026-group5/ilastik-mitocheck-run (README, docs/REFERENCE.md) and github.com/globias-2026-group5/agent-benchmark-ilastik-tracking (README; sessions/PROMPTS.md, RUNBOOK_blind_sessions.md, and each session's report).